

R Coding basics

Learning objectives

To be able to:

1. Set your working directory
2. Write comments in R.
3. Create section headers in RStudio.
4. Know how to use R as a calculator.
5. Create, overwrite and manipulate R objects.
6. Understand the basic rules for naming R objects.
7. Understand the syntax for calling R functions.
8. You can use install and load add-on R packages and call functions from these packages.

Working directory

- The **working directory** is the folder where R reads and saves files by default.

Check the current working directory

- Before setting or changing your current working directory, it would be good to know what your current working directory is. To do that, use the following command

```
# use the command below  
getwd()
```

Setting a working directory

- You can set it **manually**, through RStudio's menu, or **automatically using R code**.

Using R Code (manually)

- You can set the working directory directly in your R script or console:

```
setwd("C:/Users/ADMIN/OneDrive/JKUAT_ARBO_WATCH/r_intro")
```

Using the RStudio Menu

- To do this;

1. Go to the top menu bar.
2. Click **Session** → **Set Working Directory** → **Choose Directory...**
3. Browse to your desired folder and click **Open**.

- RStudio will automatically run a command like:

```
setwd("C:/Users/ADMIN/OneDrive/JKUAT_ARBO_WATCH/r_intro")
```

- You can also try other options available under *Set Working Directory* sub-menu.

To be able to manually set a working directory, it would be good to first create the target directories (folders). This way, one is able to direct the software using the link

Introduction

- In the last section, we learned how to use RStudio, the wonderful integrated development environment (IDE) that makes working with R much easier.
- In this section, we will learn the basics of using R itself.

R Script file

- Writing an R script for an analysis has several advantages over a graphical user interface (GUI)
 - *It provides a record of the exact approach used in an analysis*
 - *It enables the analysis to be easily reproduced and modified*
 - *It allows greater control*
- To get started, open RStudio, and open a new script with File > New File > R Script on the RStudio menu.



- Next, **save the script** with File > Save on the RStudio menu or by using the shortcut Command/Control + S . This should bring up the Save File dialog box. Save the file with a name like “r_basics”.
- Since we have already set the working directory, the file will automatically be saved under the directory (folder)
- R files are saved with an extension **.R**. For instance, the file created above should be reading **r_basics.R**

- Next, you should now **type all the code from this lesson into that script file.**

Comments

- There are two main types of text in an R script: **commands and comments**.
- A command is a line or lines of R code that instructs R to do something (e.g. `2 + 2`)
- A comment is text that is ignored by the computer. In R, we use the symbol (`#`) to denote a comment
- Anything that follows a `#` symbol (pronounced “hash” or “pound”) on a given line is a comment. Try typing out and running the code below to see this:

```
# A comment  
2 + 2 # Another comment  
# 2 + 2
```

- Since they are ignored by the computer, comments are meant for *humans*.
- They help you and others keep track of what your code is doing. Use them often!.

Question 1

True or False: both code chunks below are valid ways to comment code:?

```
# add two numbers  
2 + 2
```

```
2 + 2 # add two numbers
```

Note: See bottom of the lesson for the solutions to these questions.

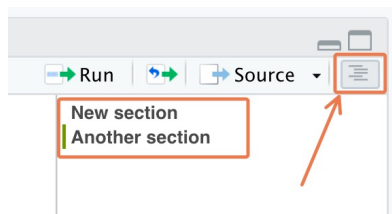
- A fantastic use of comments is to separate your scripts into sections. If you put four dashes after a comment, RStudio will create a new section in your code:

```
# New section ----
```

- Another way to insert a section is to use the menu by clicking on the menu Code-> Insert Section then enter the required details
- This has two nice benefits.
 - Firstly, you can click on the little arrow beside the section header to fold, or collapse, that section of code:

```
1 # New section ----  
2
```

- Second, you can click on the “Outline” icon at the top right of the Editor to view and navigate through all the contents in your script:



Using R as a calculator

- A basic but useful purpose of R is to perform simple calculations
- R works as a calculator, and obeys the correct order of operations. Type and run the following expressions and observe their output:

```
# Addition and Subtraction  
5 + 4
```

```
## [1] 9
```

```
124 - 26.82
```

```
## [1] 97.18
```

```
# Multiplication and Division  
5 * 4 # five times four
```

```
## [1] 20
```

```
35/8 # thirty five divide by eight
```

```
## [1] 4.375
```

```
# Exponential and Exponential Function:  
exp(-2)
```

```
## [1] 0.1353353
```

```
2 ^ 3 # two raised to the power of three
```

```
## [1] 8
```

```
2 + 2 * 2 # this is evaluated following the order of operations
```

```
## [1] 6
```

```
sqrt(100) # square root
```

```
## [1] 10
```

- The square root command shown on the last line is a good example of an R *function*, where 100 is the *argument* to the function. You will see more functions soon.

Let's remind ourselves of the keyboard short-cuts to run the R code

To **run a single line of code**, place your cursor anywhere on that line, then hit Command + Enter on macOS, or Control + Enter on Windows.

To **run multiple lines**, drag your cursor to highlight the relevant lines then again press Command/Control + Enter.

Question 2

In the following expression, which sign is evaluated first by R, the minus or the division?

```
5 - 5 / 5
```


Formatting code

- R does not care how you choose to space out your code.
- For the math operations we did above, all the following would be valid code:

```
2+2
```

```
## [1] 4
```

```
2 + 2
```

```
## [1] 4
```

```
2      +      2
```

```
## [1] 4
```

- Similarly, for the `sqrt()` function used above, any of these would be valid:

```
sqrt(100)
```

```
## [1] 10
```

```
sqrt( 100  )
```

```
## [1] 10
```

```
# you can even space the command out over multiple lines  
sqrt(
```

```
100  
)
```

```
## [1] 10
```

- But of course, you should try to space out your code in sensible ways (see more details under *Good Programming Skills*). Over time, as you read other people's code, you will learn that there are certain R *conventions* for code spacing and formatting.
- In the meantime, you can ask RStudio to help format your code for you. To do this, highlight any section of code you want to reformat, and, on the RStudio menu, go to Code > Reformat Code, or use the shortcut Shift + Command/Control + A.

Stuck on the + sign

- If you run an incomplete line of code, R will print a + sign to indicate that it is waiting for you to finish the code.
- For example, if you run the following code:

```
sqrt(100
```

you will not get the output you expect (10). Rather the console will sqrt(and a + sign:

```
> sqrt(100  
+ |
```

- R is waiting for you complete the closing parenthesis. You can complete the code and get rid of the + by

just entering the missing parenthesis:

```
)
```

```
> sqrt(100  
+ )  
[1] 10
```

- Alternatively, press the escape key, ESC while your cursor is in the console to start over.
- Now that you have been able to write your first script in R, let us save it and then **quit** the session.
- Ensure you remember where you have saved the file.
- Open RStudio once more and try to locate the script file and then open it once more for use

Objects in R

Create an object

- When you run code as we have been doing above, the result of the command (or its *value*) is simply displayed in the console—it is not stored anywhere.

```
2 + 2 # R prints this result, 4, but does not store it
```

```
## [1] 4
```

- To store a value for future use, assign it to an *object* with the *assignment operator*, `<-` :

```
my_obj <- 2 + 2 # assign the result of `2 + 2` to the object called `my_obj`  
my_obj # print my_obj
```

```
## [1] 4
```

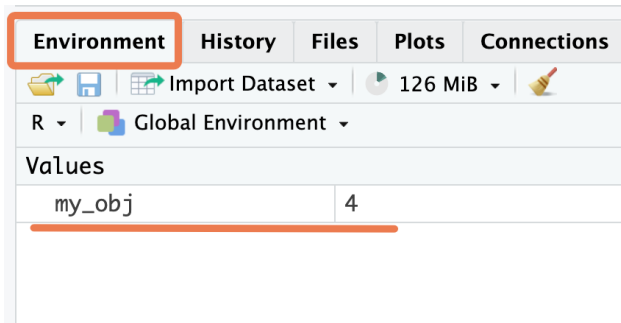
- The assignment operator, `<-` , is made of the ‘less than’ sign, `<` , and a minus, `-` . You will use it thousands of times over your R lifetime, so please don’t type it manually! Instead, use RStudio’s shortcut, **alt + -** (**alt AND minus**) on Windows or **option + -** (**option AND minus**) on macOS.

Also note that you can use the *equals* sign, `=`, for assignment.

```
my_obj = 2 + 2
```

But this is not commonly used by the R community (mostly for historical reasons), so we discourage it too. Follow the convention and use `<-`.

- Now that you've created the object `my_obj`, R knows all about it and will keep track of it during this R session. You can view any created objects in the *Environment* tab of RStudio.



What is an object?

- So what exactly is an object? Think of it as a named bucket that can contain anything. When you run the code below:

```
my_obj <- 20
```

you are telling R, “put the number 20 inside a bucket named ‘my_obj’”.

```
my_obj <- 20
```

*Put the number 20 inside
an object called `my\_obj`*



Once the code is run, we would say, in R terms, that “the value of object called my_obj is 20”.

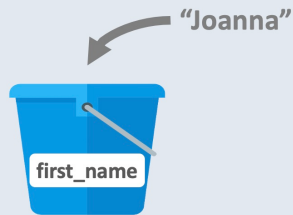
And if you run this code:

```
first_name <- "Joanna"
```

you are instructing R to “put the value ‘Joanna’ inside the bucket called ‘first_name’”.

```
first_name <- "Joanna"
```

*Put the value “Joanna” inside an object called
‘first_name’*



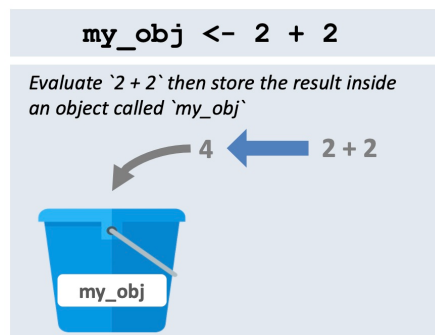
- Once the code is run, we would say, in R terms, that “the value of the first_name object is Joanna”.

- Note that R evaluates the code *before* putting it inside the bucket.

So, before when we ran this code,

```
my_obj <- 2 + 2
```

- R firsts does the calculation of $2 + 2$, then stores the result, 4, inside the object.



Question 3

Consider the code chunk below:

```
result <- 6 + 6 + 6
```

What is the value of the `result` object created?

A. $6 + 6 + 6$

B. numeric

Rename an object

- Sometimes you may want to rename an object. It is not possible to do this directly.
- To rename an object, you make a copy of the object with a new name, and delete the original.

For example, maybe we decide that the name of the `ebola_sierra_leone_data` object is too long. To change it to the shorter “`ebola_data`” run:

```
ebola_data <- ebola_sierra_leone_data
```

```
## Error: object 'ebola_sierra_leone_data' not found
```

This has copied the contents from the `ebola_sierra_leone_data` *bucket* to a new `ebola_data` *bucket*.

You can now get rid of the old `ebola_sierra_leone_data` bucket with the `rm()` function, which stands for “remove”:

```
rm(ebola_sierra_leone_data)
```

```
## Warning in rm(ebola_sierra_leone_data): object 'ebola_sierra_leone_data' not  
## found
```

Overwrite an object

- Overwriting an object is like changing the *contents* of a *bucket*.

- For example, previously we ran this code to store the value “Joanna” inside the `first_name` object:

```
first_name <- "Joanna"
```

- To change this to a different, simply re-run the line with a different value:

```
first_name <- "Luigi"
```

- You can take a look at the Environment tab to observe the change.

Working with objects

- Most of your time in R will be spent manipulating R objects. Let's see some quick examples.
- You can run simple commands on objects. For example, below we store the value 100 in an object and then take the square root of the object:

```
my_number <- 100  
sqrt(my_number)
```

```
## [1] 10
```

- R “sees” `my_number` as the number 100, and so is able to evaluate its square root.

- You can also combine existing objects to create new objects. For example, type out the code below to add `my_number` to itself, and store the result in a new object called `my_sum`:

```
my_sum <- my_number + my_number
```

- What should be the value of `my_sum`? First take a guess, then check it.

To check the value of an object, such as `my_sum`, you can type and run just the code `my_sum` in the Console or the Editor. Alternatively, you can simply highlight the value `my_sum` in the existing code and press Command/Control + Enter.

Question 4 Consider the code below. What is the value of the answer object?

```
eight <- 9  
answer <- eight - 8
```

Some errors with objects

```
first_name <- "Luigi"  
last_name <- "Fenway"
```

```
full_name <- first_name + last_name
```

```
Error in first_name + last_name : non-numeric argument to binary operator
```

- The error message tells you that these objects are not numbers and therefore cannot be added with +. This is a fairly common error type, caused by trying to do inappropriate things to your objects. Be careful about this.

In this particular case, we can use the function `paste()` to put these two objects together:

- Another error you'll get a lot is Error: object 'XXX' not found. For example:

```
my_number <- 48 # define `my_obj`  
My_number + 2 # attempt to add 2 to `my_obj`
```

```
Error: object 'My_number' not found
```

- Here, R returns an error message because we haven't created (or *defined*) the object `My_obj` yet. (Recall that R is case-sensitive.)

Question 5

The code below returns an error. Why?

```
my_first_name <- "Kene"  
my_last_name <- "Nwosu"  
my_first_name + my_last_name
```

Avoid naming your objects (variables) the same as existing R functions (**e.g., c, mean, sum**). Doing so will overwrite the function in your current session and can lead to unexpected errors or incorrect results. Always choose descriptive and unique names for your objects.

Data Types / Classes

Data Types	Stores
real	floating point numbers
integer	integers
complex	Complex numbers
factor	categorical data
character	strings
logical	TRUE or FALSE
NA	Missing
NULL	Empty
Function	Function type

Vector

- A vector can only contain objects of the same class

```
# numeric vector
a <- c(1,2,5.3,6,-2,4)
# character vector
b <- c("one","two","three")
# logical vector
c <- c(TRUE,TRUE,TRUE,FALSE,TRUE,FALSE)
```

Factors

- Used to represent categorical data.
- Can be unordered or ordered.

-A factor is like an integer vector where each integer has a label.

```
# A factor variable  
x <- factor(c("yes", "yes", "no", "yes", "no"))  
x
```

```
## [1] yes yes no  yes no  
## Levels: no yes
```

Missing Values

- Missing values are represented by the symbol **NA** (not available)
- Impossible values (e.g., dividing by zero) are represented by the symbol NaN (not a number)
- Can be unordered or ordered. -A factor is like an integer vector where each integer has a label.

Packages

- As we mentioned previously, R is wonderful because it is user extensible: anyone can create a software *package* that adds new functionality. Most of R's power comes from these packages.
- A package is a set of functions designed to perform more specific statistical or graphical tasks
- Within The Comprehensive R Archive Network (CRAN), Currently, the CRAN package repository features **22,952+** (as at 25.10.2025) available packages.
- In the previous lesson, you installed and loaded the {pacman} package using the `install.packages()` and `library()` functions. Let's learn a bit more about packages now.

pacman::p_load()

- Rather than use two separate functions, `install.packages()` then `library()`, to install then load packages, you can use a single function, `p_load()`, from the {pacman} package to automatically install a package if it is not yet installed, *and* load the package. We encourage this approach in the rest of this course.
- **pacman** is one of the most useful packages in R. The package simplifies package management by allowing you to **load, install, and update** multiple packages with a single command
- Once installed as above, it can be used as follows

- To use the package, type `pacman::p_load(package_name)` to both install and load the package

```
pacman::p_load(ggplot2, dplyr, tidyr)  
# the double colon :: (namespace operator) is used to access a specific  
# function or object from a package without loading the entire package using library()
```

- `p_load()` checks whether each listed package is installed — if not, it installs it — and then loads it automatically

Try this now for the `outbreaks` package, which we will use soon:

```
pacman::p_load(outbreaks)
```

Sometimes an RStudio feature will require a contributed package. A pop-up will ask permission to install the package the

Importing data in R

- R works with different types of datasets
- Base R functions *read.table* and *read_._csv* can read in data stored as text files, delimited by *almost anything*
- Data from other statistics tools can be read using
 - *readxl* - for excel datasets
 - *haven* - for Stata files
 - *foreign* - for other files like SPSS
- Although creating data frames from existing data structures is extremely useful, by far the most common approach is to create a data frame by importing data from an external file.
- To do this, you'll need to have your data correctly formatted and saved in a file format that R is able to recognise.
- Fortunately, R is able to recognise a wide variety of file formats, although in reality you'll probably end up only using **two or three** regularly

File format

- Before importing any data, it would be good to know the file format in which the data is saved.
- Your data could either be from Stata (.dta), or SPSS (.sav) or text file(.txt), command Separated values (.csv) or just an excel file (.xlsx)
- This will help in choosing the correct funtions to import your data

Using RStudio import dataset tool (GUI)

- We can import data into R from different softwares/ formats including Ms Excel, Stata, SPSS and others
- RStudio provides a **point-and-click interface** that is very beginner-friendly.

Steps:

1. Open RStudio → go to the **Environment** pane.

2. Click **Import Dataset** → options:

- **From Text (readr)** → CSV, TSV, or other delimited files.
- **From Excel** → XLSX or XLS files.
- **From SPSS/Stata/SAS** → requires *haven* package.

3. A dialog box opens:

- *Browse and select your file.*
- *Specify options:*
 - **Header:** Yes/No (does the first row contain column names?)
 - **Separator:** Comma, Tab, Semicolon, etc.
 - **Quote:** Usually "
 - **Strings as factors:** Usually FALSE

- You can **preview your data** before importing.

4. Click **Import**:

- The dataset is loaded into your **Environment**.
- RStudio automatically generates the **R code** used for importing, visible in the **Script window**.

```
# Example:  
mydata <- readr::read_csv("C:/Users/.../datafile.csv")
```

Tip: Copy the generated code to your scripts for reproducibility instead of using the GUI every time.

Using RStudio Files Pane

1. Navigate to the Files pane.
2. Locate your data file.
3. Click on it → RStudio shows an Import Dataset button.
4. Follow the same options as above.

Importing Excel files in RStudio

- Requires the *readxl* package:

```
library(readxl)  
my_excel <- read_excel("C:/Users/.../datafile.xlsx", sheet = 1)
```

RStudio GUI also supports Excel files and shows a preview before import.

Getting help in R

Sometimes it is tricky ...

- R can be tricky to learn,
- feel free to contact/tweet/IG/DM any of the demonstrators with questions
- *please make sure that you have thought through your question in advance*
- *R problems can be solved with judicious use of Google and StackOverFlow*

Within R: Help with functions

```
# help with a specific function
help(function_name)
?function_name

# quick reminder of the function arguments
arg(function_name)

# see an example
example(function_name)

# see the source code
## in console
function_name
## in View pane (easier to read, syntax highlighting)
View(function_name)
```

Within R: help.search

```
# when ?function_name fails (e.g. don't have package loaded)
help.search("function_name") # note quotation marks
??function_name

# for when you can't quite remember the function name
??something_like_function_name

# R help start page
help.start()

## note that using the RStudio Help home button gives even more resources
```

Within R: Help with package

```
# `help`
help(package = "package_name")
# Help panel in RStudio will give links to all documentation
# and help pages for that package

# find/browse vignettes for installed (or specific) packages
browseVignettes()
browseVignettes("package_name")

# use auto-completion in RStudio to see what functions
# are in a package
?package::
```

Additional Help Areas

- R Help Mailing List - <https://stat.ethz.ch/mailman/listinfo/r-help>
- R CookBook - <http://www.cookbook-r.com/>
- R-Bloggers - <http://www.r-bloggers.com/>
- Stack overflow About R - <http://stackoverflow.com/tags/r/info>

- Stack overflow R FAQ - <http://stackoverflow.com/tags/r>
- R google group - <https://groups.google.com/forum/#!forum/r-help-archive>
- <https://exeter-data-analytics.github.io/IntroToR/>

Good programming practice

Naming conventions

- R has no standardised naming conventions
- Always choose a naming convention to work with; for example
 - all lowercase: e.g. ***adjustcolor***
 - underscore separated: e.g. ***numeric_version***
 - lowerCamelCase: e.g. ***addTaskCallback***
 - UpperCamelCase: e.g. ***SignatureMethod***
- **Avoid SPACES** while naming files
- Strive for names that are concise and meaningful
- R file names should be meaningful and end in .R.

Object names

- Variable and function names should be lowercase.

```
# Good  
day_one  
day_1  
day.one
```

```
# Bad  
first_day_of_the_month  
DayOne  
dayone  
djm1
```

- Avoid using names of existing functions and variables.

Files organisation

- File organisation makes code and data analysis project readable
- Data should be separated from codes
- Documents should be separated from codes
- Use project facility of RStudio each time you start working on a new project

Organise the code within each file

- Start each file with a comment stating **who wrote it and when, what it contains, and how it fits into the larger program**
- Load all required packages
- Source required data files if any

```

#-----
## Analysis of the Dengue Fever data from Kilifi - Kenya
## Henry Athiany - October 2025
#-----
#-----
# 0 - Load Libraries
#-----
library(dplyr)
library(ggplot2)
#-----
# 1 - Source Data
#-----
df1 <- read_csv("data/my_data.csv")
#-----
# 2 - Start my code
#-----
mean(mtcars$mpg)

```

Syntax

- Place spaces around all infix operators (=, +, -, <-, etc.).
- Use <-, not =, for object assignment in R.
- Use comments to mark off sections of code.
- Comment your code with care. Comments should explain the why, not the what
- Each line of a comment should begin with the comment symbol and a single space
- Keep your lines less than 80 characters.

```
# This is a comment
# Good
# Object assignment in R
x <- 10

#Bad
x=10
```

```
# Good
average <- mean(feet / 12 + inches, na.rm = TRUE)

# Bad
average<-mean(feet/12+inches,na.rm=TRUE)
```

Use <-, not =, for assignment.

```
# Good
x <- 5

# Bad
x = 5
```

Good practice for R Scripts

- Organising your R script well will help you and others understand and use it.
- Add comment or two at start to describe purpose of script
 - Use one or more # to start a comment
- Load required data and packages at the start
- Avoid hard-coding: define variables such as file paths early on

- Give functions and variable meaningful names
- use `####` or `#---` to separate sections (in RStudio Code > Insert Section)

Answers

1. True.
2. The division sign is evaluated first.
3. The answer is C. The code `6 + 6 + 6` gets evaluated before it is stored in the object.
4. The value is 1. The code evaluates to `9-8`.
5. You cannot add two character strings. Adding only works for numbers.